



Campaign Tracker

Complete Project Documentation

Full-Stack Marketing Campaign Management System

Stack	Node.js + Express + MySQL + HTML/CSS/JS
Deployment	Railway (Cloud Platform)
Live URL	web-production-ef9b1.up.railway.app
GitHub	github.com/campaigntracker
Version	2.0 — Production Ready

Table of Contents

1. Project Overview	3
2. Tech Stack & Architecture	3
3. Database Schema	4
4. Backend — API Routes	5
5. Frontend Features	7
6. Security Implementation	8
7. All Features in Detail	9
8. Issues Fixed & Improvements Made	12
9. Deployment Guide	13
10. File Structure	14

1. Project Overview

Campaign Tracker is a full-stack web application designed to help marketers and businesses create, manage, and track their marketing campaigns. It provides a complete solution from campaign creation to detailed analytics, including click tracking, conversion monitoring, and revenue reporting.

What the App Does:

- **User Management:** Register, login, logout with JWT authentication. Forgot/reset password via email.
- **Campaign Management:** Create, edit, delete campaigns with status (Active/Paused/Ended) and date ranges.
- **Click Tracking:** Record clicks on campaigns with IP address and device information.
- **Conversion Tracking:** Track conversions with type and revenue data per campaign.
- **Analytics Dashboard:** View clicks, conversions, revenue, and conversion rate with visual charts.
- **Profile Management:** Update name and change password from the dashboard.
- **Mobile Responsive:** Works on all devices — desktop, tablet, and mobile phones.

2. Tech Stack & Architecture

Layer	Technology	Purpose
Backend	Node.js + Express.js	REST API server
Database	MySQL	Data storage
Authentication	JWT + bcryptjs	Secure login & tokens
Email	Nodemailer + Gmail SMTP	Password reset emails
Frontend	HTML5 + CSS3 + Vanilla JS	User interface
Charts	Chart.js v4.4.1	Analytics visualization
Deployment	Railway	Cloud hosting
Version Control	Git + GitHub	Code management

Architecture Flow:

Browser (HTML/CSS/JS) → HTTP Requests → Express.js Server → Route Handlers (auth.js / campaign.js / track.js / analytics.js / profile.js) → MySQL Database

3. Database Schema

The application uses 4 MySQL tables:

Table 1: users

Column	Type	Description
id	INT AUTO_INCREMENT PK	Unique user identifier
name	VARCHAR(100)	Full name of the user
email	VARCHAR(100) UNIQUE	Email address (used for login)
password	VARCHAR(255)	bcrypt hashed password
created_at	TIMESTAMP	Account creation date

Table 2: campaigns

Column	Type	Description
id	INT AUTO_INCREMENT PK	Unique campaign identifier
name	VARCHAR(100)	Campaign name
source	VARCHAR(100)	Traffic source (Google, Facebook etc.)
medium	VARCHAR(100)	Marketing medium (CPC, Email etc.)
budget	DECIMAL(10,2)	Campaign budget in INR
status	VARCHAR(20)	active / paused / ended
start_date	DATE NULL	Campaign start date
end_date	DATE NULL	Campaign end date
user_id	INT FK	References users(id)
created_at	TIMESTAMP	Creation timestamp

Table 3: clicks

Column	Type	Description
id	INT AUTO_INCREMENT PK	Unique click identifier
campaign_id	INT FK	References campaigns(id)
ip_address	VARCHAR(50)	IP address of the visitor
device	VARCHAR(100)	Device type (mobile/desktop)
clicked_at	TIMESTAMP	Time of the click

Table 4: conversions

Column	Type	Description
id	INT AUTO_INCREMENT PK	Unique conversion identifier
campaign_id	INT FK	References campaigns(id)
conversion_type	VARCHAR(100)	Type of conversion (Purchase, Signup etc.)
revenue	DECIMAL(10,2)	Revenue generated from conversion
converted_at	TIMESTAMP	Time of the conversion

4. Backend — API Routes

The backend is built with Express.js and organized into 5 route files. All protected routes require a valid JWT token in the Authorization header.

4.1 Authentication Routes (/auth)

Method	Endpoint	Auth	Description
POST	/auth/register	No	Register a new user account
POST	/auth/login	No	Login and receive JWT token
POST	/auth/forgot-password	No	Send password reset email
POST	/auth/reset-password	No	Reset password with token

How Authentication Works:

1. User registers with name, email, password. Password is hashed with bcrypt (10 salt rounds) before storing.
2. User logs in — server verifies password against hash, returns a JWT token signed with JWT_SECRET.
3. Token is stored in localStorage on the frontend.
4. Every protected API request sends the token as: Authorization: Bearer <token>
5. The verifyToken middleware validates the token on every protected route.

4.2 Campaign Routes (/campaign)

Method	Endpoint	Auth	Description
POST	/campaign/create	Yes	Create a new campaign
GET	/campaign/all	Yes	Get all campaigns for logged-in user
PUT	/campaign/edit/:id	Yes	Update an existing campaign
DELETE	/campaign/delete/:id	Yes	Delete a campaign

4.3 Tracking Routes (/track)

Method	Endpoint	Auth	Description
POST	/track/click	Yes	Record a click on a campaign
POST	/track/conversion	Yes	Record a conversion with revenue

4.4 Analytics Routes (/analytics)

Method	Endpoint	Auth	Description
GET	/analytics/:id	Yes	Get clicks, conversions, revenue for a campaign

The analytics query uses a SQL JOIN:

```
SELECT c.name, COUNT(DISTINCT cl.id) AS clicks, COUNT(DISTINCT cv.id) AS conversions,
IFNULL(SUM(cv.revenue),0) AS revenue FROM campaigns c LEFT JOIN clicks cl ON c.id =
cl.campaign_id LEFT JOIN conversions cv ON c.id = cv.campaign_id WHERE c.id = ? AND
```

```
c.user_id = ?
```

4.5 Profile Routes (/profile)

Method	Endpoint	Auth	Description
GET	/profile	Yes	Get current user profile info
PUT	/profile/update-name	Yes	Update user display name
PUT	/profile/update-password	Yes	Change account password

5. Frontend Features

The frontend is built with pure HTML, CSS, and Vanilla JavaScript — no frameworks required. It uses a single-page app approach where all dashboard content is loaded dynamically without page reloads.

Page	File	Description
Login	<code>login.html</code>	Email/password login with forgot password link
Register	<code>register.html</code>	New account creation with validation
Dashboard	<code>dashboard.html</code>	Main app with sidebar navigation
Forgot Password	<code>forgot.html</code>	Enter email to receive reset link
Reset Password	<code>reset.html</code>	Set new password via reset link
404 Error	<code>404.html</code>	User-friendly error page for unknown routes

Dashboard Sections (loaded dynamically):

- **Welcome Screen:** Shows real-time stats — total campaigns, clicks, conversions, and revenue fetched from API.
- **Create Campaign:** Form to create a campaign with name, source, medium, budget, status, and date range.
- **My Campaigns:** List of all campaigns with Edit and Delete buttons, status badges, and date range display.
- **Edit Campaign:** Pre-filled form to update any campaign field including status.
- **Analytics:** Dropdown to select campaign, then shows 4 metric cards + bar chart using Chart.js.
- **Profile:** Shows user avatar, name, email, join date. Allows updating name and changing password.

UI Design Highlights:

- Dark theme with CSS variables for consistent colors throughout
- Smooth animations — `slideUp`, `slideLeft`, `fadeIn`, `pulse`
- Gradient backgrounds and glowing hover effects on cards
- Mobile-responsive with a slide-in sidebar drawer on small screens
- Plus Jakarta Sans font from Google Fonts for modern typography
- Status badges with color coding: green (active), amber (paused), red (ended)

6. Security Implementation

- ✓ **JWT Authentication:** JSON Web Tokens used for all protected routes. Tokens expire in 7 days. Secret stored as environment variable `JWT_SECRET`.
- ✓ **bcrypt Password Hashing:** All passwords hashed with bcrypt using 10 salt rounds before storing in database. Plain text passwords never stored.
- ✓ **Environment Variables:** All sensitive data (`JWT_SECRET`, `MYSQLPASSWORD`, `EMAIL_USER`, `EMAIL_PASS`) stored as Railway environment variables, never in code.
- ✓ **Route Protection:** All campaign, analytics, track, and profile routes require a valid JWT token via `verifyToken` middleware.
- ✓ **User Data Isolation:** All database queries filter by `user_id` so users can only see and modify their own campaigns and data.
- ✓ **Input Validation:** All form inputs validated on both frontend and backend before processing. Empty fields, invalid emails, and negative budgets rejected.
- ✓ **Duplicate Email Prevention:** Register route checks for existing email before creating account and returns a clear error message.
- ✓ **Secure Password Reset:** Reset tokens use JWT signed with (`JWT_SECRET` + current password hash), making tokens invalid after password change.
- ✓ **Campaign Ownership Check:** Delete and edit operations verify the campaign belongs to the requesting user before making changes.
- ✓ **CORS Enabled:** Cross-Origin Resource Sharing enabled for API accessibility from the frontend domain.

7. All Features in Detail

7.1 User Registration

How it works step by step:

1. User fills in Full Name, Email, Password on register.html
2. Frontend validates: all fields required, valid email format, password minimum 6 characters
3. POST request sent to /auth/register with JSON body
4. Backend checks if email already exists in database
5. If new: password hashed with bcrypt, user inserted into users table
6. Success response returned, user redirected to login page

7.2 User Login

1. User enters email and password on login.html
2. Frontend validates fields are not empty
3. POST request sent to /auth/login
4. Backend queries database for user with matching email
5. bcrypt.compare() checks password against stored hash
6. If match: JWT token generated with user ID, expires in 7 days
7. Token stored in localStorage, user redirected to dashboard

7.3 Forgot & Reset Password

1. User clicks 'Forgot Password?' on login page, redirected to forgot.html
2. Enters email address and clicks 'Send Reset Link'
3. POST to /auth/forgot-password — server finds user by email
4. Reset token generated: JWT signed with (JWT_SECRET + user's current password hash)
5. Reset link sent to user's email via Nodemailer/Gmail SMTP
6. User clicks link in email, taken to reset.html with token in URL
7. User enters new password (min 6 chars) and confirms it
8. POST to /auth/reset-password — server verifies token is valid and not expired (1 hour)
9. New password hashed and saved, old reset tokens automatically invalidated
10. User redirected to login page after successful reset

7.4 Campaign Management

Creating a Campaign:

1. User fills: Campaign Name, Source (Google/Facebook etc.), Medium (CPC/Email etc.)
2. Sets Budget (INR), Status (Active/Paused/Ended), Start Date, End Date
3. Frontend validates all required fields and positive budget

4. POST to /campaign/create with JWT token in header
5. Server inserts campaign linked to logged-in user's ID
6. Success screen shows with campaign ID and navigation options

Editing a Campaign:

1. User clicks Edit button on a campaign card in My Campaigns
2. Edit form opens pre-filled with current campaign data
3. User modifies any fields and clicks Save Changes
4. PUT to /campaign/edit/:id — server verifies campaign ownership
5. Database updated, user returned to My Campaigns list

Deleting a Campaign:

User clicks Delete, confirms in dialog, DELETE request sent to /campaign/delete/:id. Server verifies ownership before deleting. Campaign list refreshes automatically.

7.5 Analytics

The analytics page provides comprehensive insights for each campaign using SQL aggregate queries.

- **Campaign Selector:** Dropdown populated with user's campaigns — no manual ID entry required.
- **Total Clicks:** Count of all recorded clicks for the selected campaign.
- **Total Conversions:** Count of all recorded conversions for the selected campaign.
- **Total Revenue:** Sum of all revenue from conversions, displayed in INR.
- **Conversion Rate:** Calculated as (conversions / clicks) x 100, shown as percentage.
- **Bar Chart:** Visual Chart.js bar chart showing all 4 metrics side by side with custom colors.

7.6 Profile Page

- Avatar circle showing first letter of user's name with gradient background
- Displays full name, email address, and account creation date
- Update Name — change display name with instant save
- Change Password — requires current password verification before allowing change
- Password update uses bcrypt to hash new password before storing

7.7 Mobile Responsive Design

- Viewport meta tag added to all HTML pages for proper mobile scaling
- Hamburger menu button (■) appears on screens smaller than 768px
- Sidebar becomes a slide-in drawer with dark overlay backdrop
- Stats grid switches from 4 columns to 2 columns on mobile
- Campaign cards stack vertically with full-width buttons on mobile
- All inputs and forms are touch-friendly with comfortable tap targets
- Tablet view (max 1024px) uses 2-column stats grid and narrower sidebar

8. Issues Fixed & Improvements Made

Critical Security Issues Fixed:

✗ Hardcoded JWT Secret	✓ SECRET = 'secretkey' replaced with process.env.JWT_SECRET environment variable
✗ Exposed Database Password	✓ password: 'Gokul@123' removed from db.js, now uses only environment variables
✗ Unprotected Analytics Route	✓ Added verifyToken middleware + user ownership check to /analytics/:id
✗ Unprotected Track Routes	✓ Added verifyToken + campaign ownership verification to /track/click and /track/conversion
✗ No Token Expiry	✓ JWT tokens now expire in 7 days using expiresIn: '7d' option

Warnings Fixed:

- Input validation added to all forms (register, login, create campaign, edit campaign)
- Duplicate email check added to registration — returns friendly error instead of MySQL error
- Welcome screen stats now load real data from API instead of hardcoded zeros
- Analytics page now shows campaign dropdown instead of requiring manual Campaign ID entry
- Analytics results shown as proper UI cards with charts instead of raw JSON dump

Features Added:

- Edit Campaign — full update form with all fields editable
- Campaign Status — Active/Paused/Ended with color-coded badges
- Campaign Date Range — start and end date fields
- Analytics Charts — Chart.js bar chart for visual data representation
- 404 Error Page — friendly error page for unknown routes
- Global Error Handler — Express error middleware for uncaught errors
- README.md — complete project documentation
- schema.sql — database setup script
- Mobile Responsive Design — sidebar drawer, responsive grid
- Profile Page — view and update name and password
- Forgot Password — email-based password reset flow
- Reset Password — secure token-based password reset

9. Deployment Guide

The application is deployed on Railway with a MySQL database.

Environment Variables Required:

Variable	Description	Example
JWT_SECRET	Secret key for JWT token signing	any_long_random_string
MYSQLHOST	MySQL database host	containers-us-west-xx.railway.app
MYSQLUSER	MySQL username	root
MYSQLPASSWORD	MySQL password	your_db_password
MYSQLDATABASE	Database name	campaign_tracker
MYSQLPORT	MySQL port number	6543
EMAIL_USER	Gmail address for sending emails	campaign_tracker@gmail.com
EMAIL_PASS	Gmail App Password (16 chars)	xxxx xxxx xxxx xxxx

Deployment Steps:

1. Push code to GitHub repository
2. Create new project on Railway (railway.app)
3. Connect GitHub repository to Railway
4. Add MySQL service to the Railway project
5. Add all environment variables in Railway Variables tab
6. Railway auto-deploys on every git push to main branch
7. Run schema.sql on MySQL to create required tables

Local Development Setup:

```
npm install
```

```
node campaigntracker/server.js
```

10. File Structure

```
camapigntracker/ (root folder) | +-- campaigntracker/ (main application folder) | | |
+-- server.js (Express server entry point) | +-- auth.js (Register, Login, Forgot/Reset
Password) | +-- campaign.js (Create, Read, Update, Delete campaigns) | +-- track.js
(Click and conversion tracking) | +-- analytics.js (Analytics data aggregation) | +--
profile.js (User profile management) | +-- mailer.js (Nodemailer email configuration) |
+-- db.js (MySQL database connection) | | | +-- dashboard.html (Main dashboard page) |
+-- login.html (Login page) | +-- register.html (Registration page) | +-- forgot.html
(Forgot password page) | +-- reset.html (Reset password page) | +-- 404.html (404 error
page) | | | +-- script.js (All frontend JavaScript logic) | +-- style.css (All styles
and responsive CSS) | +-- schema.sql (Database setup SQL script) | +-- README.md
(Project documentation) +-- package.json (Node.js dependencies) +-- package-lock.json
(Dependency lock file)
```

Key Dependencies (package.json):

Package	Version	Purpose
express	^4.x	Web server framework
mysql2	^3.x	MySQL database driver
bcryptjs	^2.x	Password hashing
jsonwebtoken	^9.x	JWT token generation and verification
nodemailer	^6.x	Email sending (password reset)
cors	^2.x	Cross-Origin Resource Sharing

Project Summary

Metric	Count
Total API Routes	14
Database Tables	4
HTML Pages	6
Critical Issues Fixed	4
Warnings Resolved	6
Features Added	12
Lines of JavaScript	~600+
Lines of CSS	~500+

Campaign Tracker is a production-ready full-stack application built with security, scalability, and user experience in mind. It demonstrates proper JWT authentication, database design, RESTful API architecture, responsive UI design, and cloud deployment.

Live URL: <https://web-production-ef9b1.up.railway.app/campaigntracker/login.html>

GitHub: <https://github.com/gokulsingh2/campaigntracker>

GitHub: <https://github.com/RISHI1104RISHABH/campaigntracker-main>

